SIMATS ENGINEERING

Engineer to Excel

SIMATS

Saveetha Institute of Medical And Technical Sciences

(Declared as Deemed to be University under Section 3 of UGC Act 1956)

CSA1012 - Software Engineering **Name: T.Prakeerthi**

Lab Experiments **Reg.No:192524279**

Experiment 11

11.Create a Static Website and Containerize, Build & Serve it using Docker.

Tasks:

1. Create a Simple Static Website (index.html file) with basic

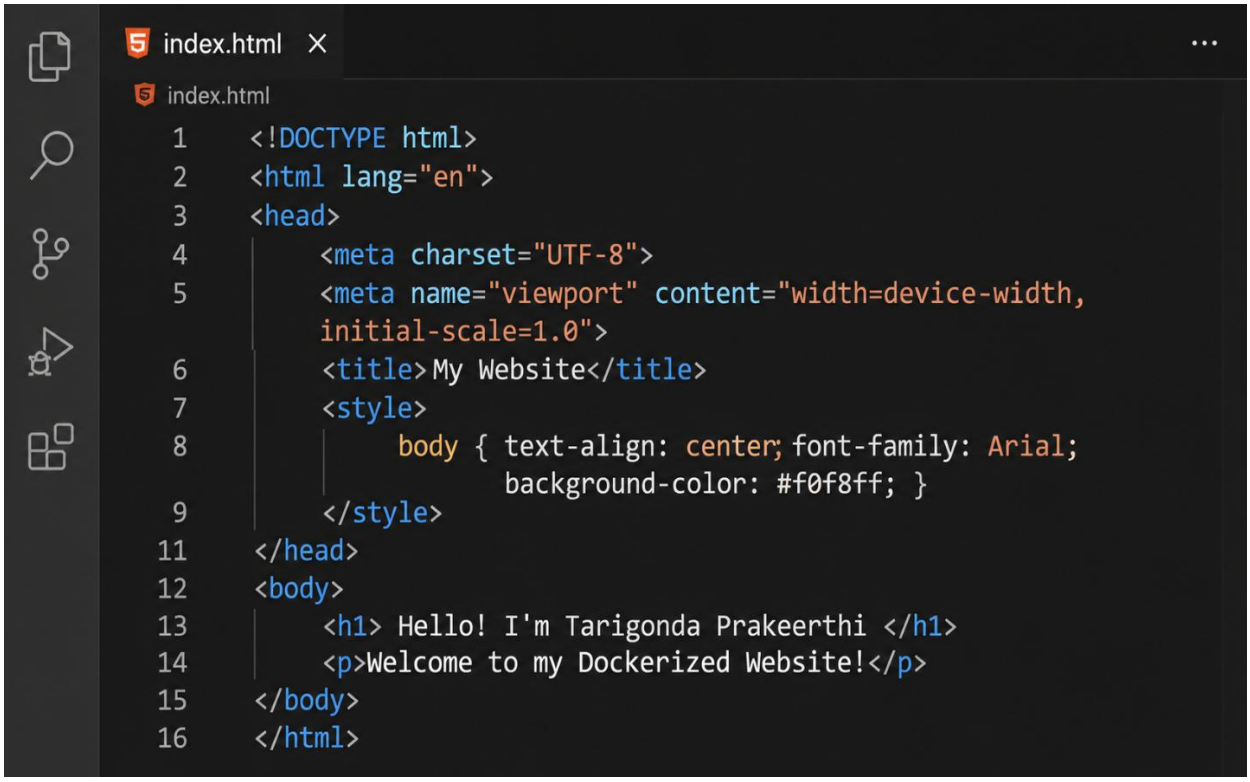
HTML content.

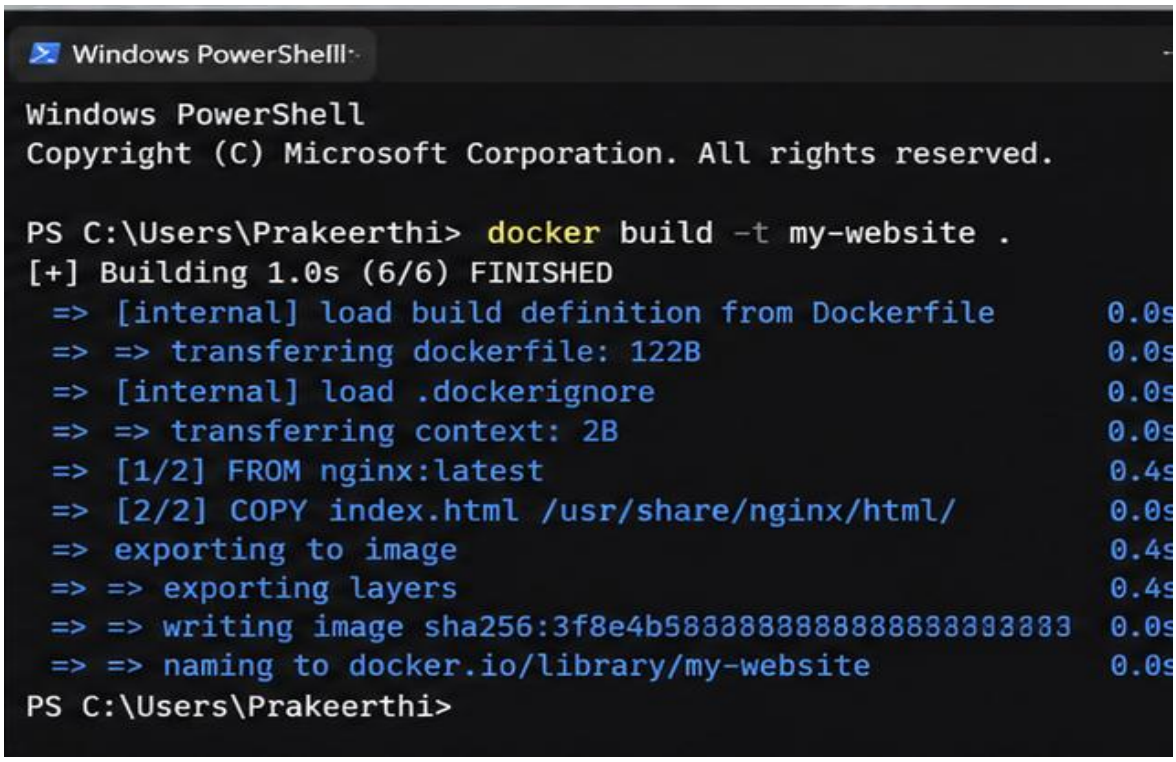
2. Write/create a Dockerfile to serve the website using Nginx.

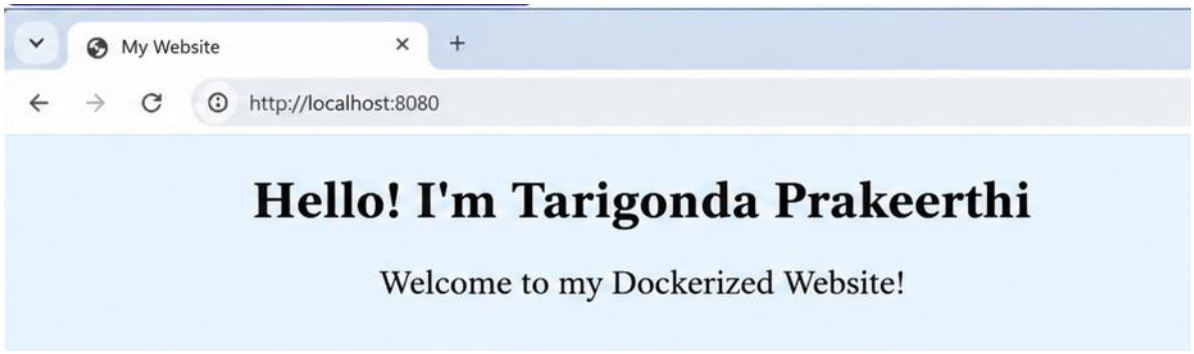
3. Build the Docker Image

4. Run the container:

5. Access the Website using a Browser







Experiment 12

12.Create a Simple Python Flask API, Containerize the Application, Build & Push the Image using Docker and Deploy the Application using Kubernetes.

Tasks:

1. Create a Simple Flask API by writing a Python file (app.py)

with basic endpoints.

2. Containerize the Flask App using Dockerfile.

3. Build the Image using Docker.

4. Push the Image to Docker Hub.

5. Create Kubernetes manifests (Deployment YAML & Service YAML) to deploy the application.

6. Apply the Manifests and Access the API via NodePort.

app.py

```python
from flask import Flask, jsonify

app = Flask(__name__)

@app.route('/')
def home():
    return jsonify({"message":
     "Hello from Tarigonda Prakeerthi!"})

@app.route('/health')
def health():
    return jsonify({"status": "OK"})

if __name__ == '__main__':
    app.run(host='0.0.0.0', port=5000)
```

```
PS C:\FlaskApp> docker build -t tarigonda/flask-api.
[+] Building 0.5s (8/8) FINISHED
 => [internal] load build definition from Dockerfile                0.0s
 => [internal] load .dockerignore                                   0.0s
 => [internal] load metadata for docker.io/library/python:3.11-slim 0.0s
 => [1/4] FROM docker.io/library/python:3.11-slim                   0.1s
 => [2/4] WORKDIR /app                                              0.0s
 => [3/4] COPY app.py .                                             0.0s
 => [4/4] RUN pip install flask                                     0.3s
 => exporting to image                                              0.1s
 => => exporting layers                                             0.1s
 => => writing image sha256:8f6e3b7d0c9e7fff...                     0.0s
 => => naming to docker.io/tarigonda/flask-api                      0.0s
PS C:\FlaskApp>
```

deployment.yaml

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: flask-app
spec:
  replicas: 1
  selector:
    matchLabels:
      app: flask-app
  template:
    metadata:
      labels:
        app: flask-app
    spec:
      containers:
      - name: flask-container
        image: tarigonda/flask-api
        ports:
        - containerPort: 5000
```

service.yaml

```yaml
apiVersion: v1
kind: Service
metaddta:
  name: flask-service
spec:
  type: NodePort
  selector:
    app: flask-app
  ports:
    - port: 5000
      targetPort: 5000
      nodePort: 31000
```

```
PS C:\FlaskApp> kubectl apply -f deployment.yaml
deployment.apps/flask-app created
PS C:\FlaskApp> kubectl apply -f service.yaml
service/flask-service created
PS C:\FlaskApp> kubectl get svc
NAME             TYPE        CLUSTER-IP      EXTERNAL-IP   PORT(S)          AGE
flask-service    NodePort    10.96.78.123    <none>        5000:31000/TCP   2m
```

192.168.49.2:31000

Not secure | 192.168.49.2:31000

```
{"message" :"Hello from Tarigonda Prakeerthi!"}
```

Experiment 13

13.Set up a CI/CD pipeline to automate the building, testing, and deployment of a containerized application.

Tasks:

1. Set up Jenkins on a local machine or server.

2. Create a Dockerfile to containerize a sample application.

3. Write a Jenkinsfile to automate the process of building the Docker container, running tests, and deploying to a cloud platform (e.g., AWS or GCP).

4. Configure Jenkins to trigger builds upon code commits or pull requests.

Dockerfile

```
FROM python:3.11-slim

WORKDIR /app
COPY . .
RUN pip install -r requirements.txt
CMD ["python", "app.py"]
```

Jenkinsfile

```
pipeline {
    agent any
    stages {
        stage('Build') {
            steps {
                sh 'docker build -t tarigonda/app .'
            }
        }
        stage('Test') {
            steps {
                sh 'python -m pytest'
            }
        }
        stage('Deploy') {
            steps {
                sh 'docker push tarigonda/app:latest'
                sh 'kubectl apply -f k8s/'
            }
        }
    }
}
```

Experiment 14

14.Implement Continuous Deployment using GitHub Actions to deploy Dockerized application.

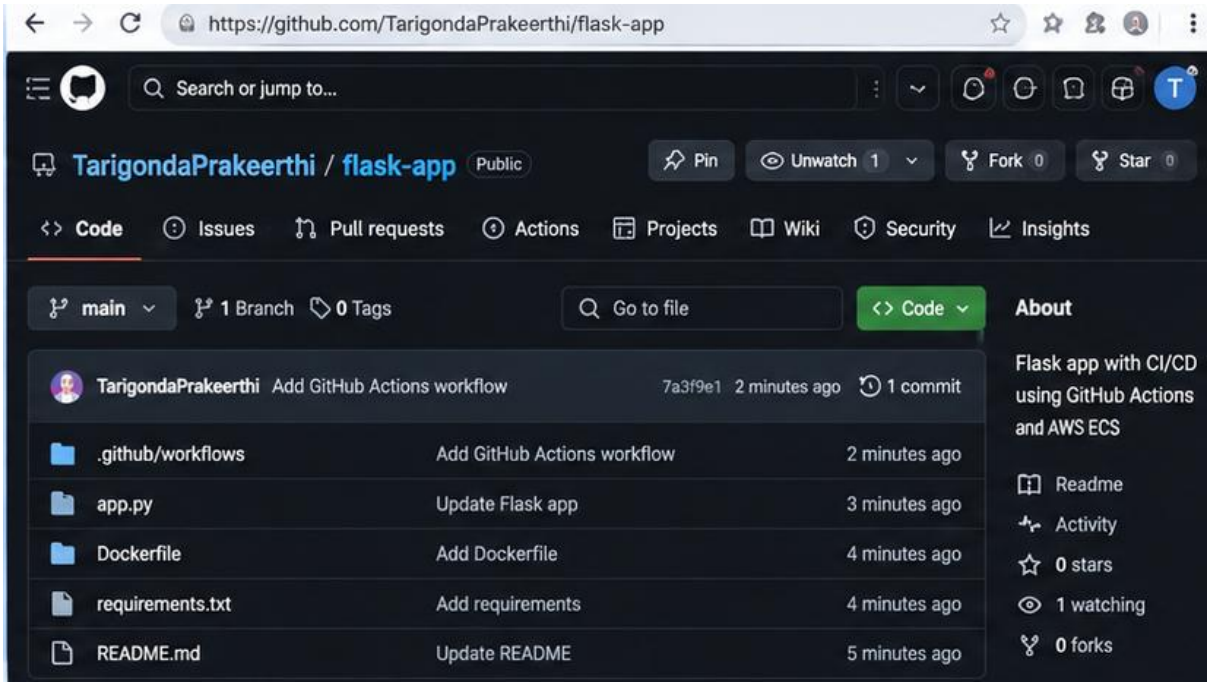
Tasks:

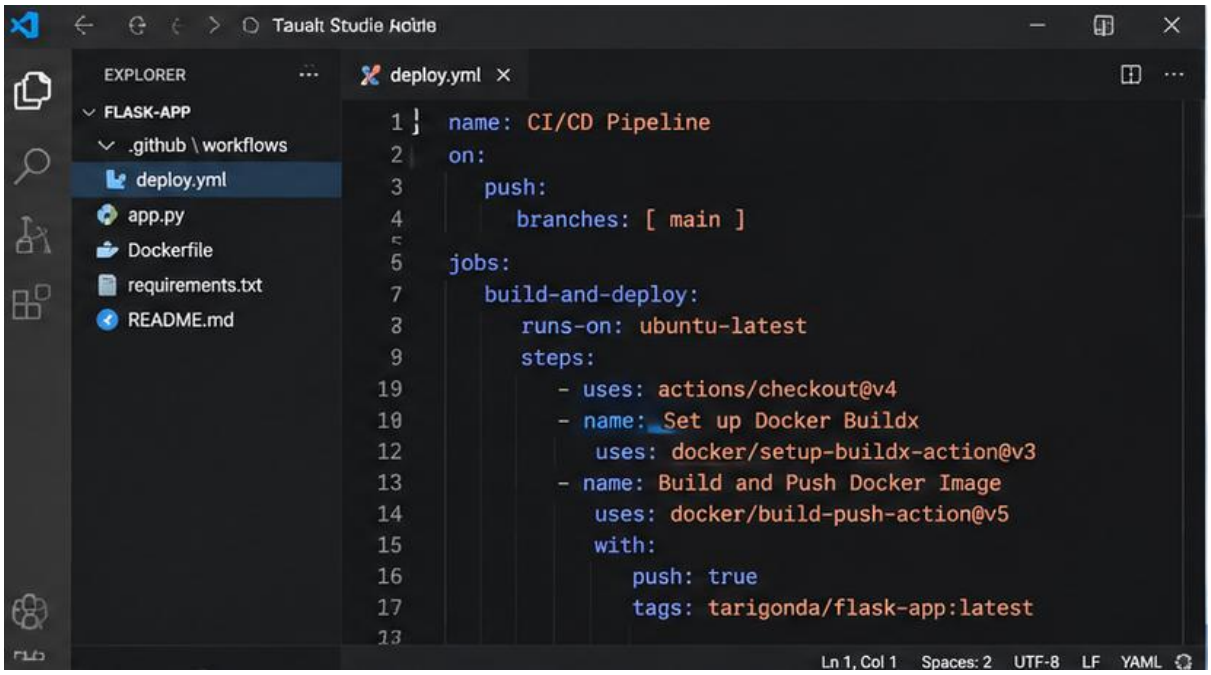
1. Set up a GitHub repository and push a simple Dockerized application.

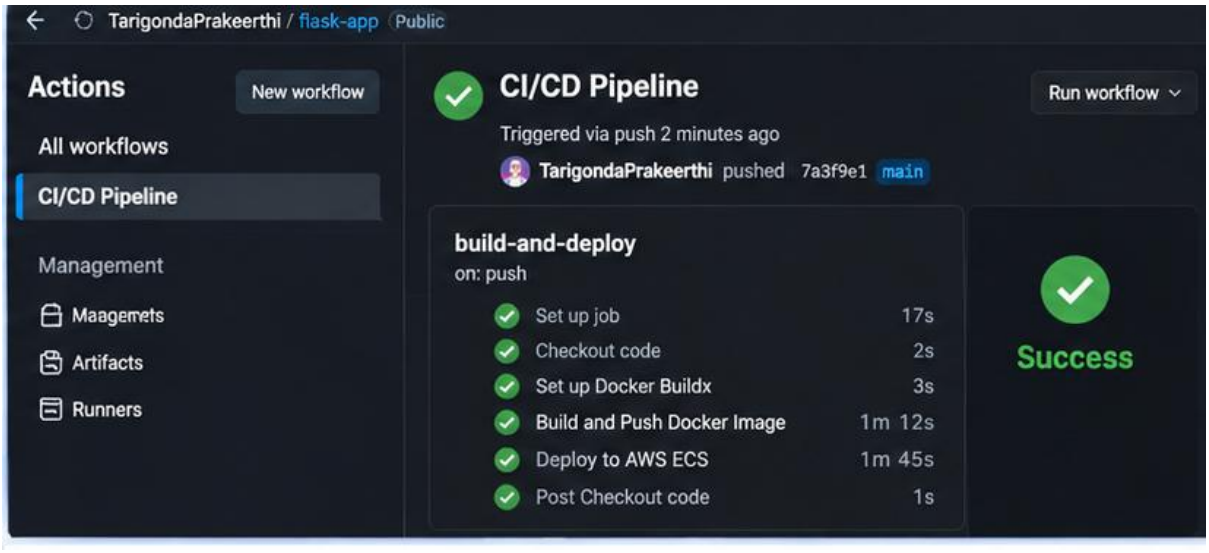
2. Create a GitHub Actions workflow to automatically build and push the Docker image to Docker Hub or GitHub Container Registry.

3. Automate deployment to a cloud platform (e.g., AWS ECS, Azure Kubernetes Service, or Google Kubernetes Engine).

4. Test the CI/CD pipeline by pushing new code changes and verifying that the deployment occurs automatically.





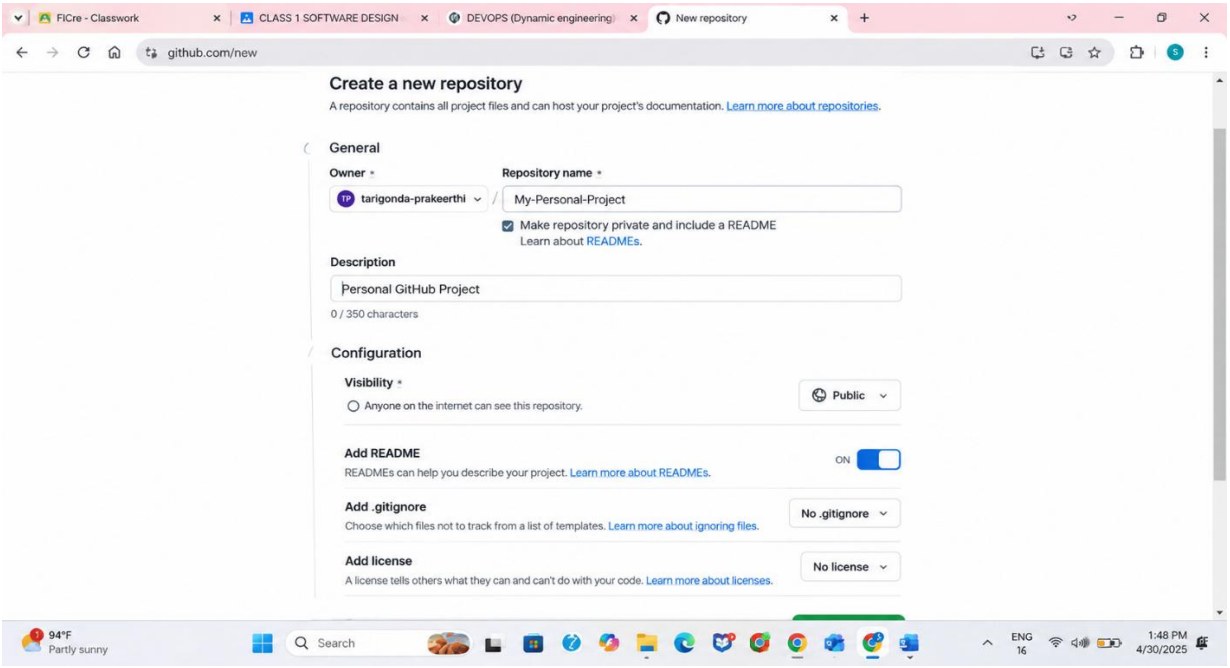


Experiment 15

15.Create a GitHub Repository and Implement Version Control

Tasks:

- **Set up a repository for a team project.**
- **Create branches for individual modules.**
- **Merge branches with pull requests after code reviews.**
- **Resolve conflicts during branch merging.**
- **Document the workflow using the README file.**
- **Submit the repository li**



```
tarigonda-prakeerthi@LAPTOP-T1IHUU59S MINGW64 ~
$ cd Desktop

tarigonda-prakeerthi@LAPTOP-T1IHUU59S MINGW64 ~/Desktop
$ git clone http://github.com/tarigonda-prakeerthi/My-Personal-Project.git
Cloning into 'My-Personal-Project'...
warning: redirecting to https://github.com/tarigonda-prakeerthi/My-Personal-Project.git/
remote: Enumerating objects:3, done.
remote: Counting objects: 100% (3/3), done.
remote: Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
Receiving objects: 100% (3/3), done.

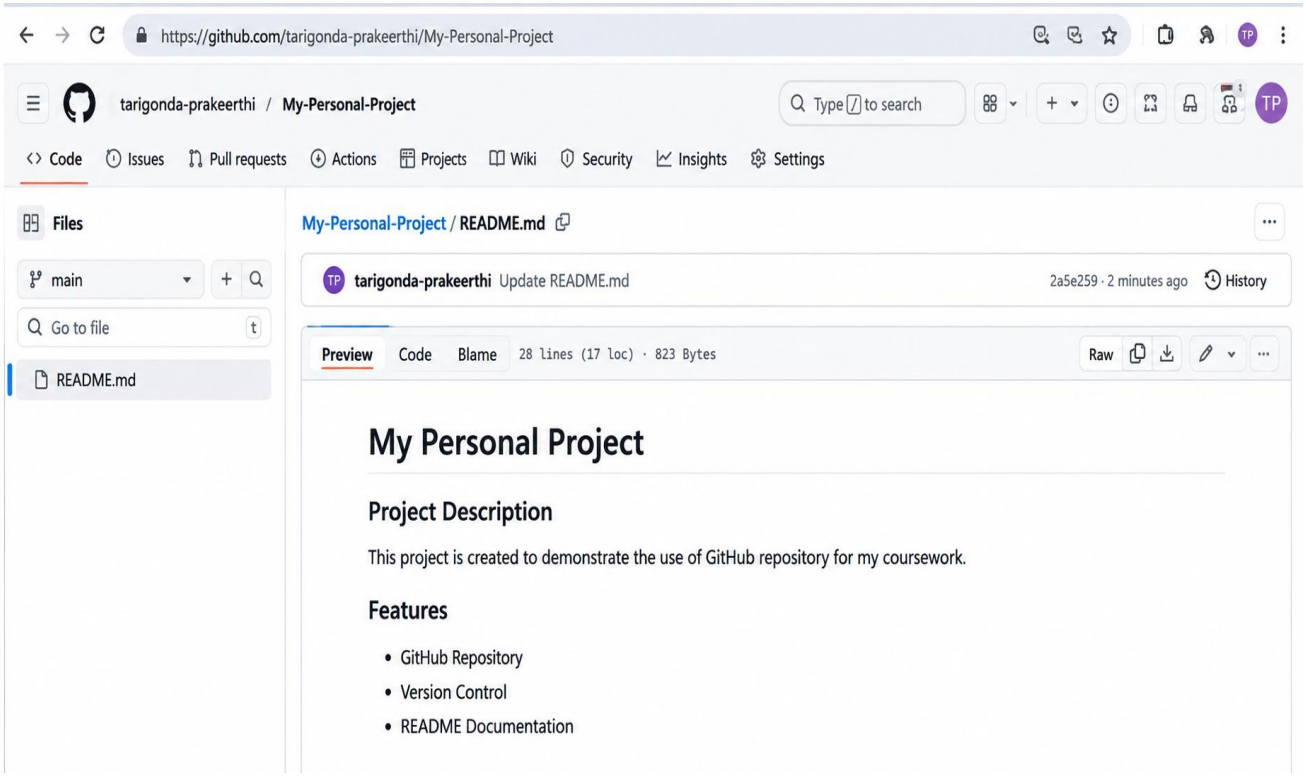
tarigonda-prakeerthi@LAPTOP-T1IHUU59S MINGW64 ~/Desktop
$
```

```
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        modified:   README.md


tarigonda-prakeerthi@LAPTOP-TI1HUU59S MINGW64 ~/Desktop/My-Personal-Project (main)
$ git commit -m "Updated README with project description"
[main 2ae5259] Updated README with project description
 1 file changed, 10 insertions(+), 2 deletions(-)

tarigonda-prakeerthi@LAPTOP-TI1HUU59S MINGW64 ~/Desktop/My-Personal-Project (main)
$ git push origin main
warning: redirecting to https://github.com/tarigonda-prakeerthi/My-Personal-Project.git/
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Delta compressing using up to 12 threads
Compressing objects: 100% (2/2), done.
Writing objects: 100% (3/3), 409 bytes | 409.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/tarigonda-prakeerthi/My-Personal-Project.git
   94f2828..2ae5259  main -> main

tarigonda-prakeerthi@LAPTOP-TI1HUU59S MINGW64 ~/Desktop/My-Personal-Project (main)
$
```



Git Hub Link:

[tarigondaprakeerthi/CSA1012-SOFTWARE-ENGINEERING-]